

Chapter 6 Homework

6.3

For each of the variables `a, b, c, d, e` in this C program, say whether the variable should be kept in memory or a register, and why.

```
int f(int a, int b)
{
    int c[3], d, e;
    d = a + 1;
    e = g(c, &b);
    return e + c[1] + b;
}
```

解答：

- `a`：寄存器（或直接参与指令计算后不再保留）。`a` 只在 `d = a + 1` 中使用一次，不取地址，也不需要跨调用 `g` 保活。
- `b`：内存。因为有 `&b`，必须有可寻址的栈位置；并且 `g(c, &b)` 可能修改 `b`，返回后要从内存读取最新值。
- `c`：内存。`c` 是数组并以地址形式传给 `g`，需要连续可寻址存储，不能只放寄存器。
- `d`：寄存器（甚至可被优化消除）。`d` 不取地址，且后续未使用，属于短生命周期临时值。
- `e`：寄存器。`e` 接收 `g` 的返回值后立即参与返回表达式，通常可保存在寄存器中。

结论：应放内存的是 `b`、`c`；可放寄存器的是 `a`、`d`、`e`（其中 `d` 可被死代码删除）。

6.7

A display is a data structure that may be used as an alternative to static links for maintaining access to nonlocal variables. It is an array of frame pointers, indexed by static nesting depth. Element `D_i` of the display always points to the most recently called function whose static nesting depth is `i`.

In Program 6.3, function `prettyprint` is at depth 1, `write` and `show` are at depth 2, and so on.

a. Show the sequence of machine instructions required to fetch the variable `output` into a register at line 14 of Program 6.3, using static links.

b. Show the machine instructions required if a display were used instead.

解答：

设：

- FP：当前活动记录（这里是 indent，深度 3）的帧指针
- SL：静态链字段在帧中的偏移
- OFF_output：变量 output 在 prettyprint（深度 1）帧中的偏移
- R1、R2：通用寄存器
- DISPLAY：display 数组基址
- WORD：机器字节宽度

a) 使用静态链（static links）

indent（深度 3）访问 output（深度 1）要沿静态链走两步：indent → show → prettyprint。

```
MOV R1, FP           ; R1 = 当前帧( indent )
LOAD R1, [R1 + SL]    ; R1 = show 的帧指针 (深度 2)
LOAD R1, [R1 + SL]    ; R1 = prettyprint 的帧指针 (深度 1)
LOAD R2, [R1 + OFF_output] ; R2 = output
```

b) 使用 display

display 直接保存“每个静态深度最近一次调用”的帧指针，所以可直接取 D_1。

```
LOAD R1, [DISPLAY + 1*WORD] ; R1 = D1 = prettyprint 的帧指针
LOAD R2, [R1 + OFF_output]  ; R2 = output
```

对比：静态链需要按层逐级解引用；display 用一次索引即可到目标层，访问非局部变量指令更短。